Homework 3: Robot Manipulation and Semantic Robot Knowledge

Human-centered Intelligent System Lab
National Yang Ming Chiao Tung University

Fall 2026

Introduction

In this homework you will connect symbolic robot knowledge to the mathematics and execution of manipulation. You first represent robot specifications and FK/IK executions as structured RDF and validate them with SHACL. You then implement forward and inverse kinematics for a simulated six-degree-of-freedom UR5 arm. Finally, your solvers drive a deterministic finite-state-machine (FSM) pick-and-place pipeline in PyBullet [1].

The semantic layer deliberately comes first. Task 1 uses executions produced by the TA reference solvers, so it does not depend on your FK or IK code. Once your solvers exist, the same grounding and validation layer gives immediate, machine-readable feedback on their executions.

1 Learning Objectives

After completing the assignment, you should be able to:

1. ground robot specifications and kinematic executions as structured RDF using CORA, SOMA, IEEE 1872 POS, and QUDT vocabularies;
2. author SHACL constraints, including a SHACL–SPARQL comparison across nodes, and interpret validation reports [3];
3. compose classic Denavit–Hartenberg transforms and construct a 6×6 geometric Jacobian [2];
4. implement and tune iterative Jacobian-based inverse kinematics; and
5. diagnose kinematic failures within a complete manipulation FSM.

2 Assignment at a Glance

Task	Work	Files you edit	Points
1	Semantic grounding and SHACL validation	`semantic/ground_execution.py`, `semantic/shapes.ttl`	30
2	Forward kinematics and Jacobian	`fk.py`	20
3	Iterative inverse kinematics	`ik.py`	40
4	FSM manipulation integration (no code changes)	None	10
	Total		**100**

Students modify exactly the four files listed above. Treat all other files as provided infrastructure.

3 Environment Setup

The verified environment is Linux with Python 3.7, PyBullet, NumPy, SciPy, and JDK 11 or newer. A GPU is not used.

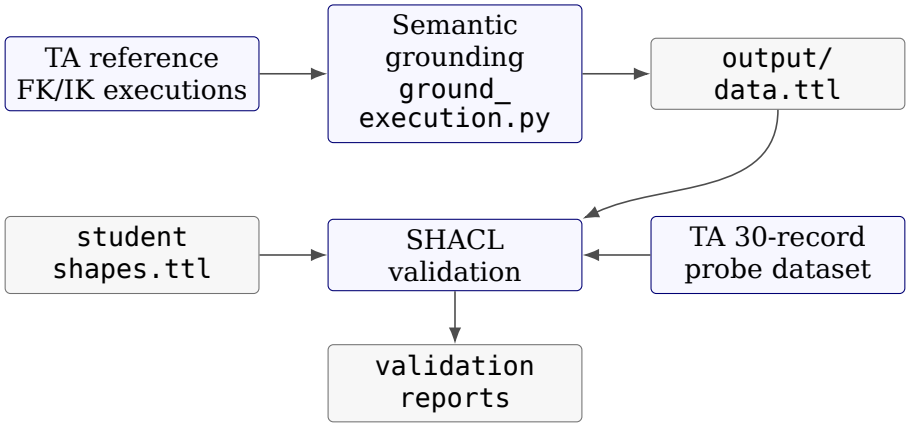
```
cd hw3_template
conda create --name taica-hw3 python=3.7 -y
conda activate taica-hw3
pip install pybullet numpy scipy
sudo apt-get -y install openjdk-11-jdk
java -version && javac -version
```

Task 1 downloads Apache Jena 4.10.0 (approximately 30 MB) on its first run and caches it under `semantic/.cache/`. Network access is not needed after the cache and Python environment are prepared. No dataset or learned model is downloaded.

4 Task 1: Semantic Grounding and SHACL Validation (30 points)

4.1 Execution model and workflow

Convert numerical FK/IK execution records into a semantic graph, then use SHACL to detect problem states directly from the stored numbers:



The design follows the REUSE principle: a robot is a `cora:Robot`; joints and joint states use SOMA; poses use `soma:6DPose` and IEEE 1872 POS; units use QUDT IRIs. The local `hw3:` vocabulary is only for concepts not covered by those standards. Numeric values must be structured nodes and typed `xsd:double` literals, never JSON strings.

4.2 Mathematical representation

Let an RDF data graph be a finite set of triples

$$G \subseteq \mathcal{N} \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L}),$$

where $\mathcal{N}$, $\mathcal{P}$, and $\mathcal{L}$ are the sets of nodes, properties, and literals. We use the predicate notation

$$C(x) \iff (x, \mathsf{rdf{:}type}, C) \in G, \qquad p(x, y) \iff (x, p, y) \in G.$$

Grounding is a serialization function

$$\gamma : \mathcal{E} \longrightarrow 2^{G}, \qquad G_{\text{exec}} = G_{\text{ontology}} \cup \bigcup_{e \in \mathcal{E}} \gamma(e),$$

which maps each robot specification or execution record e to RDF triples. It must preserve structure: a six-joint configuration c satisfies

$$\mathsf{JointConfiguration}(c) \ \wedge \ |\{s : \mathsf{hasJointState}(c, s)\}| = 6,$$

and every linked state carries its joint identity and numerical angle:

$$\forall s \, [\mathsf{hasJointState}(c, s) \Rightarrow \exists j, v \ (\mathsf{isStateOfJoint}(s, j) \wedge \mathsf{hasJointPosition}(s, v))].$$

A grounded IK execution x similarly connects one robot, target, output configuration, status, residual r_x, and target distance d_x.

For shapes graph S and data graph G, define validation as

$$\mathrm{Report}(S, G) = \{(x, m, p, v) : x \text{ violates a constraint with message } m \text{ on path } p \text{ at value } v\}.$$

The graph conforms exactly when

$$\mathrm{conforms}(S, G) \iff \mathrm{Report}(S, G) = \emptyset.$$

The four numerical course constraints are

$$\begin{aligned}
\Phi_{\mathrm{FK}}: &\quad e_x \le 0.005, && x : \mathsf{FKComputation},\\
\Phi_{\mathrm{AR}}: &\quad d_x \le 0.90, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{NC}}: &\quad r_x \le 0.02, && x : \mathsf{IKComputation},\\
\Phi_{\mathrm{JL}}: &\quad \ell_j \le q_s \le u_j, && s : \mathsf{JointState},\ \mathsf{isStateOfJoint}(s, j).
\end{aligned}$$

Violation produces, respectively, `FK_INACCURATE`, `ARM_OUT_OF_RANGE`, `NO_CONVERGENCE`, or `JOINT_LIMIT_VIOLATION`. All inequalities are inclusive: equality with a threshold or joint bound conforms.

4.3 Grounding requirements

Read the worked example `fk_computation_to_triples()` and the serializers in `semantic/common.py`. Implement:

1. `robot_spec_to_triples()`: create `stu:my_ur5` as a `cora:Robot`, record its producer and six DoF, and link six `soma:RevoluteJoint` instances. Each joint must contain its index, D–H parameters a, d, α, and inclusive lower/upper limits.
2. `ik_computation_to_triples()`: create one `hw3:IKComputation` with provenance, robot, target, status, residual, target distance, and a structured six-state joint configuration. Use `common.joint_config_to_ttl()`.

4.4 SHACL requirements

The provided FK accuracy shape is a worked example: pose error must be at most 0.005 m. Add the following three shapes:

Condition	**Legal range**	**Required message prefix**
Target distance	$d \le 0.90$ m	`ARM_OUT_OF_RANGE:`
IK residual	$r \le 0.02$ m	`NO_CONVERGENCE:`
Joint position	$q_{\min} \le q \le q_{\max}$	`JOINT_LIMIT_VIOLATION:`

Use `sh:maxInclusive` for the first two constraints. The third must be a SHACL–SPARQL constraint on `soma:JointState`, because its position and the linked joint's bounds occur on different nodes. Its filter must use strict `<` and `>` comparisons; equality with a bound is legal.

4.5 Run and expected result

```
bash semantic/run_task1.sh --group <your-group-id>
# after Tasks 2 and 3:
bash semantic/run_task1.sh --group <your-group-id> --own
```

The command checks toolchains, prepares Jena, grounds the data, runs three validations, and scores the result. A correct implementation has no grounding structure violations, flags `target_mid` and `target_far` as out of range, leaves `target_near` clean, reports no joint-limit violation in the reference run, and matches all 30 records in `ta-answer-key.json`. The expected Task 1 score is 30/30.

5 Task 2: Forward Kinematics (20 points)

Implement `your_fk(DH_params,q,base_pos)` in `fk.py`. Its inputs are the six classic D–H parameter records, six joint angles, and a base position. It returns the seven-dimensional end-effector pose $[x, y, z, q_x, q_y, q_z, q_w]$ and the 6×6 geometric Jacobian.

For each revolute joint, use the classic D–H transform

$$^{i-1}T_i = R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i).$$

Save the preceding-frame origin p_{i-1} and z axis z_{i-1}. For end-effector position p_e, Jacobian column i is

$$J_i = \begin{bmatrix} z_{i-1} \times (p_e - p_{i-1}) \\ z_{i-1} \end{bmatrix}.$$

The D–H table in `get_ur5_DH_params()` is authoritative and intentionally differs slightly from the official UR5 specification. Do not use PyBullet or another library to directly compute FK/Jacobians, and do not modify the final `adjustment` block.

```
python fk.py              # visible easy / medium / hard tests
python fk.py -g -vp       # optional GUI and pose visualization
```

Success means zero FK and Jacobian error counts for every visible test file. The post-score semantic diagnosis should report conforming sampled records.

6 Task 3: Inverse Kinematics (40 points)

Implement `your_ik()` in `ik.py`. Warm-start from the current joint configuration, repeatedly call your own FK/Jacobian, and reduce a six-vector containing position and orientation error. A robust recommended update is damped least squares:

$$\Delta q = J^T \left(J J^T + \lambda^2 I \right)^{-1} \Delta x, \qquad q \leftarrow \mathrm{clip}(q + s\Delta q, q_{\min}, q_{\max}),$$

where λ is damping and s is the step rate. A rotation vector of $R_{\text{target}} R_{\text{current}}^T$ provides a useful orientation error. Stop when the error norm is below the threshold or the iteration budget is exhausted.

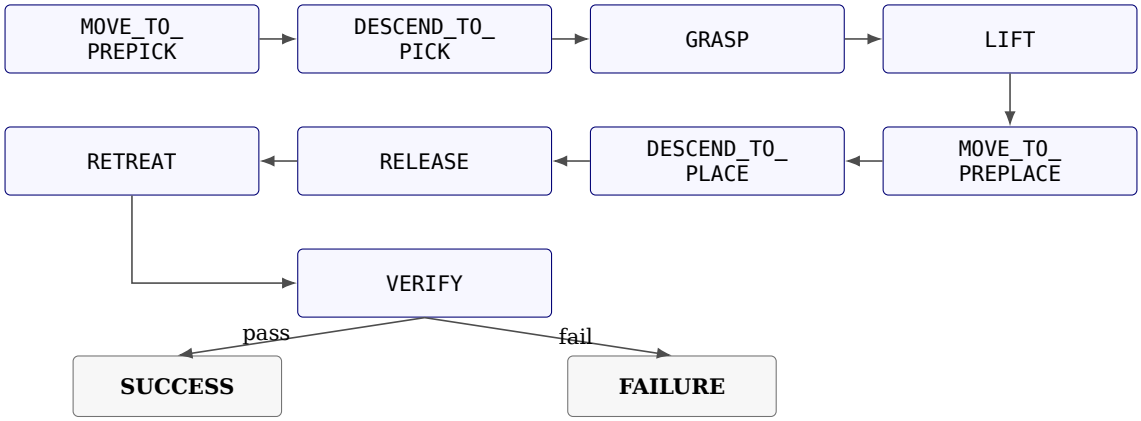
The implementation must not call PyBullet IK or another library's IK solver. `pybullet_ik()` is for behavioral comparison only. Grading invokes your function with default arguments, so tune the defaults rather than relying on command-line changes.

```
python ik.py
```

Aim for zero error counts on the easy, medium, and hard cases, mean positional error near 10^{-3} m, and conforming semantic diagnosis records.

7 Task 4: FSM Manipulation Pipeline (10 points)

No code change is required. The provided FSM runs ten seeded pick-and-place episodes through



Every motion state solves the target using your IK and requires the achieved end-effector position to be within 3 cm. It also evaluates your FK on measured joint angles and requires agreement with the simulator within 1 cm. The pick descent continues until suction contact; verification requires the block to rest within 6 cm of the goal.

```
python fsm_task.py
python fsm_task.py -g       # optional PyBullet visualization
```

The expected result is ten `SUCCESS` episodes and 10/10 points. Failures name both the FSM state and reason, such as `IK_MISS`, `FK_MISMATCH`, `NO_CONTACT`, or `PLACE_MISS`; the semantic layer additionally reports applicable problem flags.

8 Grading and Submission

Task 1: grounding/SHACL (S1 12 + S2 8 + S3 10)	30
Task 2: FK 10 + Jacobian 10	20
Task 3: IK hidden tests	40
Task 4: ten FSM episodes	10
Total	**100**

Submit:

1. `semantic/ground_execution.py`;
2. `semantic/shapes.ttl`;
3. `fk.py`;
4. `ik.py`; and
5. one PDF report following the required format below.

Required PDF report format

Name the file `<group-id>_hw3_report.pdf`. Begin with a cover block that lists the course, homework title, group ID, every member's student ID and name, and submission date. After the cover, use exactly the following four numbered top-level sections so that implementation, evidence, and analysis can be graded consistently.

1. **Semantic Grounding and SHACL Validation**

1.1. **Grounding design.** Explain how `robot_spec_to_triples()` and `ik_computation_to_triples()` map robot/execution data to structured RDF. Name the reused CORA, SOMA, POS, and QUDT terms, and explain why numerical values are typed `xsd:double` literals rather than JSON strings.

1.2. **SHACL constraints.** State the target class, property path, inclusive threshold, and required flag for arm range, convergence, and joint limits. Explain why the joint-limit rule requires SHACL–SPARQL.

1.3. **Execution evidence.** Insert one readable screenshot of the complete final output from `bashsemantic/run_task1.sh--group<group-id>`. The screenshot must show S1, all four S2 checks, S3 faulty/clean counts, and the 30-point total. Do not crop away the command or score summary.

1.4. **Validation analysis.** Summarize `output/validation.ttl`: identify the focus nodes and flags, explain why `target_mid` and `target_far` are out of range, and explain why `target_near` conforms. Then summarize `output/probe-validation.ttl`: report faulty/clean agreement and discuss at least one threshold-boundary record and one joint-limit record. Distinguish false positives from false negatives if any mismatch remains.

2. Forward Kinematics and Geometric Jacobian

2.1. **Method.** Describe the classic D–H transform order, how the six transforms are accumulated, how the final pose and quaternion are obtained, and how the base position and protected adjustment are used.

2.2. **Jacobian derivation.** Include the formula $J_{v,i} = z_{i-1} \times (p_e - p_{i-1})$, $J_{\omega,i} = z_{i-1}$, and explain why preceding-frame axes/origins must be stored.

2.3. **Execution evidence.** Insert the complete final `pythonfk.py` score screenshot showing every test file, FK error counts, Jacobian error counts, total out of 20, and semantic diagnosis.

2.4. **Result analysis.** Provide a compact table with each test-file name, FK error count/score, and Jacobian error count/score. If any case fails, report its measured norm, threshold (0.005 or 0.05), likely cause, and attempted correction.

3. Inverse Kinematics

3.1. **Method.** Explain the iterative update, six-dimensional pose error, rotation-vector orientation error, damped least-squares equation, warm start, stopping condition, and joint-limit clipping.

3.2. **Parameters.** State the final default damping, step rate, maximum iterations, and stopping threshold. Explain how each affects convergence and why the selected values were used.

3.3. **Execution evidence.** Insert the complete final `pythonik.py` screenshot showing each test file, mean error, error count, score, total out of 40, and semantic diagnosis.

3.4. **Result and debugging analysis.** Tabulate each test file's mean error, error count, and score. Describe at least one difficulty encountered (for example oscillation, singularity, orientation error, or joint clipping), the evidence used to diagnose it, and the final fix.

4. FSM Manipulation Pipeline

4.1. **Pipeline explanation.** Explain the pick, grasp, transport, place, release, retreat, and verification stages. State where the student's IK controls motion and where FK is cross-checked against the simulator.

4.2. **Execution evidence.** Insert the complete final `pythonfsm_task.py` screenshot showing all ten episode results, total success count, score out of 10, and semantic diagnosis. Multiple screenshots are allowed if one image cannot remain legible.

4.3. **Result analysis.** Report the number of successes and failures and, for every failed episode, its first failed state and exact reason (`IK_MISS`, `FK_MISMATCH`, `NO_CONTACT`, `GRASP_FAILED`, or `PLACE_MISS`). Explain the relationship between numerical tolerances, semantic flags, and observed manipulation behavior.

4.4. **Conclusion.** Summarize whether the semantic layer, FK, IK, and FSM agree end to end. Identify one remaining limitation or concrete improvement even when all tests pass.

Formatting rules. Use selectable text for explanations; screenshots alone are not a report. Number and caption every figure and table, refer to each from the text, keep terminal text legible at normal zoom, include units on all numerical values, and do not paste entire source files. Short relevant snippets are acceptable when accompanied by an explanation. Every screenshot must come from the final submitted implementation and visibly include the command and result.

Do not submit generated `semantic/output/`, `semantic/store/`, or `semantic/.cache/` directories.

9 Troubleshooting Checklist

- For Task 1 structure errors, inspect `semantic/output/ta-validation.ttl`; verify types, cardinalities, and `xsd:double` literals.
- For probe mismatches, use inclusive thresholds and preserve the exact message prefixes.
- If Jena download fails, extract Jena 4.10.0 manually and set `JENA_HOME`.
- For FK failures, verify classic D–H order and collect preceding-frame axes before each transform.
- For unstable IK, increase damping, lower the step rate, use rotation-vector error, and clip joint limits.
- If GUI mode fails, run headless or verify the `DISPLAY` setting.

References

[1] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. GitHub repository, 2021.

[2] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Prentice Hall, 3 edition, 2005.

[3] Holger Knublauch and Dimitris Kontokostas. Shapes constraint language (shacl). W3c recommendation, World Wide Web Consortium, July 2017.